Source Known Identifiers: A Three-Tier Identity System for Distributed Applications

Duran Serkan Kılıç[1]

[1]**Independent Researcher, Ankara, Türkiye**

Corresponding author:
Duran Serkan Kılıç[1]

Email address: duran.serkan@outlook.com

ABSTRACT

Distributed applications need identifiers that satisfy storage efficiency, chronological sortability, origin metadata embedding, zero-lookup verifiability, confidentiality for external consumers, and multi-century addressability. Based on our literature survey, no existing scheme provides all six of these identifier properties within a unified system.

This paper introduces Source Known Identifiers (SKIDs), a three-tier identity system that projects a single entity identity across trust boundaries, addressing all six properties. The first tier, Source Known ID (SKID), is a 64-bit signed integer embedding a timestamp with a 250-millisecond precision, application topology, and a per-entity-type sequence counter. It serves as the database primary key, providing compact storage (8 bytes) and natural B-tree ordering for optimized database indexing. The second tier, Source Known Entity ID (SKEID), extends the SKID into a 128-bit Universally Unique Identifier (UUID) compatible value by adding an entity type discriminator, an epoch selector, and a BLAKE3 keyed message authentication code (MAC). SKEIDs enable zero-lookup verification of identifier origin, integrity, and entity type within trusted environments, with a big-endian byte layout that preserves chronological ordering in lexicographic UUID string comparisons. The third tier, Secure SKEID, encrypts the entire SKEID using AES-256 symmetric encryption as a single-block pseudorandom permutation, producing ciphertext indistinguishable from random bytes while remaining compatible with standard UUID data-type parsers in string representation. Deterministic bidirectional transformations connect all three tiers.

The design employs a defense-in-depth security architecture with multiple independent verification layers: AES-256 encryption, BLAKE3 keyed MAC, marker byte detection, entity type matching, and record existence probability. A collision guard mechanism using variant byte iteration with cryptographic backward verification prevents misclassification between encrypted and unencrypted forms. An epoch addressing system spans approximately 17,421 years of total coverage across 256 configurable epochs.

A reference implementation in C#/.NET 10 is provided in the open-source DRN.Framework. BenchmarkDotNet measurements of the reference implementation (120 iterations, 262,144 invocations per iteration) on Apple M2 hardware show three performance tiers. Secure SKEID generation at 544.0 ± 5.7 ns takes approximately 1.4 times as long as UUID Version 7 at 377.5 ± 3.2 ns (a trade-off for providing AES-256 encryption). SKEID generation at 230.3 ± 3.5 ns is approximately 1.6 times as fast as UUID Version 7 despite embedding metadata and a BLAKE3 MAC in the same 128-bit footprint. SKID generation at 35.3 ± 1.2 ns is more than 10 times as fast as UUID Version 7 due to deterministic bit-packing without cryptographic random number generation. All ± values denote 99.9% Confidence Interval (CI) error margins.

INTRODUCTION

Problem Statement

An *entity* is a domain object with a distinct identity that persists across its lifecycle (Evans, 2003). Its *entity identifier* is the value that uniquely distinguishes it from all other entities of the same type. While this terminology is established in Domain-Driven Design, the underlying requirement of uniquely and persistently identifying records across distributed systems is universal regardless of architectural style, design patterns, or definitions. Specifically, modern distributed applications require entity identifiers that serve multiple roles simultaneously such as database primary keys, inter-application correlation tokens, and externally visible resource handles. However, existing identifier schemes force a choice between conflicting properties.

Database sequences offer compact storage (4 or 8 bytes) and natural ordering but expose creation patterns and cannot be safely passed to external consumers. UUID Version 4 (Davis, Peabody & Leach, 2024) provides 122 bits of randomness and global uniqueness but sacrifices ordering. UUID Version 7 (Davis, Peabody & Leach, 2024) restores time-ordering with a 48-bit Unix timestamp but exposes creation patterns. Snowflake-style identifiers (Twitter Engineering, 2010) embed timestamp and worker topology in 64 bits but again expose creation patterns and lose multi-century addressability.

None of these schemes offer a mechanism for zero-lookup verification. Chronological sortability and confidentiality are also inherently contradictory properties, and dual-identifier alternatives (e.g., integer primary key paired with UUID external-facing identifier) lose storage efficiency by maintaining two separate identifier columns and their associated indexes.

A multi-tier identifier system can address these challenges by providing different identifier properties for different trust levels. The proposed solution defines the following identifier properties as desired and achievable in a multi-tier distributed identity system.

1. Storage efficiency
2. Chronological sortability
3. Origin metadata embedding
4. Zero-lookup verifiability
5. Confidentiality for external consumers
6. Multi-century addressability

Table 1 summarizes identifier property satisfaction across existing schemes and the SKID system.

Table 1: Identifier property satisfaction across schemes.

Property	SKID System	UUID V4	UUID V7	Snowflake	ULID	CUID2	KSUID	DB Sequence
Storage efficiency	8/16 B	16 B	16 B	8 B	16 B	24 chars	20 B	4/8 B
Chronological sortability	Yes	No	Yes	Yes	Yes	No	Yes	Yes
Origin metadata embedding	Yes	No	No	Partial	No	No	No	No
Zero-lookup verifiability	Yes (MAC)	No	No	No	No	No	No	No
Confidentiality (external)	Encryption	Randomness	No	No	No	Hashing	No	No
Multi-century addressability	Yes (SKEID)	N/A	Yes	No	Yes	N/A	Partial	N/A

Table 2 provides a detailed feature comparison of these and additional identifier schemes.

Motivation

The necessity for a unified, multi-tier identifier system arises from the conflicting architectural requirements of modern distributed applications. Data in these systems continuously flows across persistence, internal, and external trust tiers. Each tier imposes different constraints on identifier design. Existing schemes and dual-identifier patterns force systems to compromise at least one of these requirements.

At the persistence tier, B-tree index performance demands sequential or time-ordered primary keys (Davis, Peabody & Leach, 2024, sec. 2.1 and 6.13). However, exposing these identifiers through external APIs creates an attack surface for Insecure Direct Object Reference (IDOR) vulnerabilities (MITRE Corporation, 2025a). Exposure also enables adversarial inference of generation velocity and record volume (the German Tank Problem (Ruggles & Brodie, 1947)). As a workaround, the dual-identifier pattern with an integer primary key alongside a random UUID alternate key compromises storage efficiency.

The full cost of such workarounds extends beyond the identifier columns themselves. For instance, a conventional schema using an auto-increment primary key (8 bytes), a UUID external identifier (16 bytes), and a `created_at` timestamp (8 bytes) requires 32 bytes of column data per record. In a fully indexed schema, each column demands a separate B-tree index, resulting in three indexes and their associated maintenance operations such as vacuum, reindex, and statistics collection. An identifier that embeds a timestamp and derives a UUID-compatible external representation deterministically at application runtime consolidates all three concerns into a single 8-byte primary key column with a single index. Under this fully indexed baseline, this is a 75% reduction in per-record column overhead and a reduction from three single-column indexes to one. This calculation accounts only for the single-column indexes on each field. Additional composite indexes involving these fields would amplify the savings further.

A downstream application receiving an identifier must verify its authenticity (MITRE Corporation, 2025b). Without embedded verification metadata, this validation requires a query or cache lookup per identifier. An identifier carrying a MAC would eliminate I/O-bound validation overhead and enable zero-lookup verification.

Decentralized identifier generation across globally distributed networks demands chronologically sortable (Davis, Peabody & Leach, 2024) identifiers. When data from independent generators must be merged years or decades after creation, the identifier scheme must guarantee sort-order consistency beyond the operational lifespan of individual system components. This is especially critical for any system requiring long-term data retention and historical analysis.

These conflicting trust-boundary constraints motivate a multi-tier identifier system where a single entity identity is projected to satisfy each trust boundary.

Contributions

This paper makes the following contributions.

1. **Design of a three-tier identity system** that satisfies all six desired identifier properties through a layered architecture aligned with trust boundaries (database, trusted internal, and external).
2. **A defense-in-depth security architecture** combining AES-256 encryption, BLAKE3 keyed MAC, marker byte detection, entity type matching, and a novel collision guard mechanism with cryptographic backward verification.
3. **An epoch addressing system** spanning approximately 17,421 years of total coverage through 256 configurable epochs of 2^{31} seconds each.
4. **A reference implementation** in C#/.NET 10 with open-source code, integration and unit test coverage, and BenchmarkDotNet performance data demonstrating competitive or superior performance compared to standard UUID generation.

Related Work

Distributed identifier schemes have evolved considerably since the original UUID specification. RFC 9562 (Davis, Peabody & Leach, 2024) standardized UUID versions 1, 3–8, with Version 7 introducing timestamp-ordered UUIDs that address the B-tree fragmentation problems of random Version 4 UUIDs. UUID Version 7 uses a 48-bit Unix timestamp prefix followed by random bits, providing millisecond-precision ordering and global uniqueness.

Twitter's Snowflake architecture (Twitter Engineering, 2010) pioneered timestamp-prefixed, worker-partitioned 64-bit identifiers for high-throughput systems. Snowflake identifiers embed a 41-bit timestamp with millisecond precision, a 10-bit machine identifier, and a 12-bit sequence number, enabling approximately 4,096 identifiers per millisecond per machine. Its custom epoch starts from November 4, 2010 and extends to July 10, 2080 (~69.7 years).

ULIDs (Feerasta, 2016) provide a 128-bit value composed of a 48-bit timestamp with millisecond precision. Its Unix epoch starts from January 1, 1970 and extends to ~10889 CE (~8,919 years). ULIDs are encoded as 26-character Crockford Base32 strings that sort lexicographically by creation time, targeting environments where string-based identifiers are standard.

The original CUID (Elliott, 2012) was a collision-resistant identifier specification that used a k-sortable, timestamp-prefixed structure. It was deprecated by its author due to security concerns. The CUID deprecation notice warns that "all monotonically increasing (auto-increment, k-sortable), and timestamp-based ids share the security issues with Cuid" and further states that "UUID V6-V8 are also insecure because they leak information which could be used to exploit systems or violate user privacy" (Elliott, 2012).

CUID2 (Elliott, 2022), the successor to CUID, takes a fundamentally different approach. CUID2 intentionally removed timestamps from the identifier for security reasons and instead recommends a separate `createdAt` column for time-based sorting, adding per-record storage overhead that timestamp-embedding schemes avoid. CUID2 generates identifiers by using independent entropy sources then hashing the concatenation with SHA3. This produces identifiers with strong collision resistance but relies on probabilistic uniqueness rather than deterministic construction.

KSUIDs (Segment, 2017) (K-Sortable Unique Identifier) provides a 160-bit (20-byte) value composed of a 32-bit timestamp. It has 1-second precision, a custom epoch from May 13, 2014 to June 19, 2150, and a 128-bit cryptographically random payload. KSUIDs are encoded as 27-character Base62 strings that sort lexicographically by creation time. The 128-bit random payload provides stronger collision resistance than UUID V4's 122 random bits. The string-first design targets application-layer identifiers rather than database primary keys where compact binary representation is critical for B-tree performance.

None of these approaches offers integrity verification. Table 2 presents a detailed feature comparison.

Table 2: Feature comparison of distributed identifier schemes.

Feature	SKID System	UUID V4	UUID V7	Snowflake	ULID	CUID2	KSUID	DB Sequence
Chronological ordering	250ms	No	ms	ms	ms	No	second	Numeric
Embedded timestamp	Yes	No	Yes	Yes	Yes	No	Yes	No
Multi-century addressing	Yes (SKEID)	N/A	Yes	No	Yes	N/A	Partial	N/A
Entity type	Yes (8-bit)	No	No	No	No	No	No	No
Integrity check	Yes (BLAKE3 MAC)	No	No	No	No	No	No	No
Confidentiality	Encryption (AES-256)	Randomness	No	No	No	Hashing	No	No
Zero-lookup validation	Yes	No	No	No	No	No	No	No
DB primary key size	8 B (SKID)	16 B	16 B	8 B	16 B	24 chars	20 B	4/8 B
External ID size	16 B (SKEID)	16 B	16 B	8 B	26 chars	24 chars	27 chars	4/8 B
UUID compatible	Yes (SKEID, UUID V8)	Yes	Yes	No	No	No	No	No
Key rotation	Yes (key-ring)	N/A	N/A	N/A	N/A	N/A	N/A	N/A

METHODS

Three-Tier Identity Model

SKIDs address the identifier gap through a three-tier identity model aligned with three trust boundaries commonly found in distributed architectures.

1. **Database Tier:** A 64-bit Source Known ID (SKID) serves as the primary key. It embeds a 250-millisecond-precision timestamp, application topology (application identifier, instance identifier), and a per-entity-type sequence counter, providing natural ordering and compact B-tree indexing.

2. **Trusted Environment Tier:** A 128-bit Source Known Entity ID (SKEID) encodes the SKID, the entity type, epoch metadata, a keyed MAC for integrity, and RFC 9562 Version 8 compatible markers. Applications can validate an identifier's origin, entity type, and integrity without a database lookup.

3. **External Tier:** A 128-bit Secure Source Known Entity ID (Secure SKEID) encrypts the entire SKEID block using AES-256, preventing information leakage to untrusted consumers while remaining compatible with standard UUID data-type parsers in string representation (see Limitation 6).

The three tiers are not three separate identifiers but one entity identity projected for different trust boundaries. The database stores the compact SKID, trusted applications derive the SKEID on demand, and external consumers receive the Secure SKEID. Deterministic bidirectional transformations between tiers allow any representation to be converted to any other, given the appropriate cryptographic keys. These transformations are sub-microsecond CPU operations (217–544 ns, see Performance Results) requiring no I/O or external lookup. Figure 1 illustrates the data transformations between the three tiers.

```
  Database              Trusted Internal          External / Public
 ┌──────────┐          ┌────────────────┐        ┌────────────────┐
 │ SKID     │ Generate │ SKEID          │ ToSecure │ Secure SKEID   │
 │ Sortable │──────────▶│ BLAKE3 MAC     │──────────▶│ AES Encrypted  │
 │ 64-bit   │◀──────────│ 128-bit        │◀──────────│ 128-bit        │
 └──────────┘   Parse   └────────────────┘  ToPlain  └────────────────┘
```

Figure 1: Three-tier identity model: bidirectional data transformations.

Source Known ID (SKID) 64-bit Specification

Bit Layout

A SKID is a 64-bit signed integer with the following field layout (Figure 2), packed most-significant bit first.

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|S|                    Timestamp (T – 32 bits)
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
 T| App ID (7)  | Inst (6)  |        SequenceId (18 bits)       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

Figure 2: SKID 64-bit field layout (IETF RFC diagram convention).
Total: 1 (Sign) + 32 (Timestamp) + 7 (App ID) + 6 (App Instance Id) + 18 (Sequence Id) = 64 bits.

Table 3 details the corresponding fields.

Table 3: SKID field definitions.

Field	Bit(s)	Width	Purpose
Sign / Epoch half	63	1 bit	Epoch half indicator for smooth epoch transitions
Timestamp	62–31	32 bits	250ms tick precision, epoch-relative (approximately 68 years per epoch)
App Id	30–24	7 bits	Application identifier (maximum 127)
App Instance Id	23–18	6 bits	Instance discriminator (maximum 63)
Sequence	17–0	18 bits	Per-instance monotonic counter (262,144 per 250ms tick; 1,048,576 per second)

The sign bit controls epoch half selection. When set to 1, the resulting value is negative, covering the first approximately 34 years of the epoch. When set to 0, the value becomes positive, extending coverage for the second approximately 34 years. Together, the two halves span approximately 68 years per epoch while preserving monotonic sort order in signed 64-bit representation, since negative values sort before positive values. This timestamp-leading layout produces a chronologically sortable SKID optimized for internal database indexing and ordering.

The 32-bit timestamp field stores unsigned 250-millisecond ticks elapsed since the configured epoch, giving four ticks per second. Sub-second ordering at 250ms granularity eliminates coarse-grained temporal ambiguity while preserving throughput within a compact 64-bit layout. The 18-bit sequence counter allows 262,144 unique identifiers per 250ms tick (1,048,576 per second) per instance. Implementations must reset the sequence when the timestamp advances. If the sequence is exhausted within a single tick, the generator must wait for the next tick before issuing new identifiers, applying backpressure to maintain uniqueness guarantees.

Generation Algorithm

The SKID generation procedure operates as follows. Given `entityType`, `appId`, `appInstanceId`, and a configured `epoch`:

1. Compute `elapsedTicks` as the floor of 250-millisecond ticks elapsed since the epoch (four ticks per second).
2. Assert that the elapsed ticks are within the epoch range (0 to $2^{33} - 1$ ticks, covering both epoch halves).
3. Determine the epoch half:
    1. If `elapsedTicks` $< 2^{32}$, the current half is the first half (sign bit = 1, producing a negative SKID). Otherwise, the current half is the second half (sign bit = 0, producing a positive SKID).
    2. Compute `timestamp` = `elapsedTicks` mod 2^{32} (equivalently, `elapsedTicks` AND $2^{32} - 1$).
4. Obtain the next sequence value from the per-entity-type atomic counter (resetting on timestamp change).
5. Pack fields into a 64-bit signed integer: sign bit at position 63, timestamp at positions 62–31, App ID at positions 30–24, App Instance ID at positions 23–18, and Sequence ID at positions 17–0.
6. Return the packed value.

Clock Drift Protection

The system handles backward time jumps at two levels. Minor drifts freeze the timestamp until the wall clock catches up, allowing the sequence counter to continue advancing within the frozen tick. The reference implementation uses 5 seconds as freeze threshold. Drifts beyond freeze threshold are considered as critical and force the application instance to shut down and restart with a new instance ID. This dual-threshold mechanism prevents duplicate or out-of-order identifiers without requiring external coordination.

Source Known Entity ID (SKEID) 128-bit Specification

Byte Layout

An SKEID occupies 16 bytes (128 bits), structured as shown in Figure 3.

```
Byte:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
     +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
     |EP| SKID Upper|S0|VE|ET|VA|S1|S2|S3|    MAC    |
     +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

Figure 3: SKEID 16-byte field layout (big-endian, RFC 9562 network byte order).

Table 4 details the byte-level layout of the SKEID fields.

Table 4: SKEID byte layout.

Field	Byte(s)	Width	Purpose
Epoch	0	8 bits	Epoch index (highest lexicographic priority)
SKID upper half	1–4	32 bits	SKID bits 63–32, sign-toggled for lexicographic order
SKID low byte 0	5	8 bits	MSB of SKID lower half (timestamp LSB, App ID)
Version marker	6	8 bits	`0x8D` (UUID V8, RFC 9562 Section 5.8 octet 6)
Entity Type	7	8 bits	Domain entity classification
Variant marker	8	8 bits	`0x8D` (RFC 9562 Section 4.1 octet 8 variant)
SKID low bytes	9–11	24 bits	Remaining SKID lower half
MAC	12–15	32 bits	BLAKE3 keyed MAC (contiguous)

All fields are encoded in big-endian (network byte order) per RFC 9562. The epoch byte occupies byte 0 to ensure that higher epoch values sort lexicographically after lower epoch values, regardless of timestamp. The upper SKID half at bytes 1–4 is sign-toggled (XOR with `0x80000000`) before encoding. This converts the signed SKID sort order (negative before positive) to unsigned byte sort order, preserving chronological ordering in lexicographic comparisons across both epoch halves. The most significant byte of the lower SKID half occupies byte 5, immediately after the upper half, ensuring that the timestamp least-significant bit and leading application topology bits participate in lexicographic comparison before the entity type at byte 7. The remaining lower SKID half is split around the variant marker at byte 8, with bytes 9–11 holding the last three bytes. Since the version marker, entity type, and variant marker are constant for all plain SKEIDs of a given type, lexicographic comparison of the split lower half operates correctly.

The marker bytes at positions 6 and 8 serve dual purposes.

1. **UUID V8 compatibility** makes the SKEID a valid RFC 9562 UUID.
2. **Detection signal** enables the parser to distinguish plaintext SKEIDs from encrypted Secure SKEIDs.

MAC Computation

Integrity verification relies on BLAKE3 keyed MAC (O'Connor et al., 2020). The computation procedure clears the four MAC bytes (positions 12–15) to zero, then computes a 4-byte BLAKE3 keyed MAC over the full 16-byte buffer. The resulting 4 bytes are written into the contiguous MAC positions (bytes 12–15).

BLAKE3 was selected for performance and security margin. Our BenchmarkDotNet measurements show BLAKE3 keyed hashing is approximately 3.5 times faster than HMAC-SHA-256 on small inputs (see supplementary file `supplementary-hash-benchmark-report.html`).

Generated MAC is truncated to 32 bits, offering a $1/2^{32}$ false-positive rate. This truncation is acceptable within the defense-in-depth architecture where the MAC is one of multiple verification layers, not the sole security mechanism.

The epoch byte (byte 0) is not cleared before MAC computation. It participates in the MAC input, adding tamper-resistance for the epoch field.

Generation Algorithm

Given a 64-bit SKID, epoch, entity type, and MAC key, the SKEID generation procedure:

1. Allocate a 16-byte buffer initialized to zero.
2. Split the SKID into upper and lower 32-bit halves. Toggle the most significant bit of the upper half (XOR with `0x80000000`) and write it in big-endian to bytes 1–4.
3. Write the most significant byte of the SKID lower half to byte 5.
4. Write the epoch byte to byte 0, the version marker `0x8D` to byte 6 (RFC 9562 Section 5.8 octet 6), entity type to byte 7, and variant marker to byte 8 (RFC 9562 Section 4.1 octet 8).
5. Write the remaining three bytes of the SKID lower half to bytes 9–11.
6. Compute the BLAKE3 keyed MAC and write it to bytes 12–15.
7. Construct the UUID from the 16-byte buffer, interpreting all fields in big-endian (network byte order) per RFC 9562, and return the result.

Auto-Detection Parse Logic

The parse algorithm automatically determines whether a UUID is a plaintext SKEID, a Secure SKEID, or an unrecognized value.

1. Convert the UUID to a 16-byte big-endian byte array (network byte order, per RFC 9562).
2. Check for exact primary marker bytes (`0x8D` at position 6, `0x8D` at position 8) in the byte array.
3. If both exact markers are present: attempt plaintext verification. If MAC verification succeeds, return as plaintext SKEID.
4. If exact markers are absent or plaintext verification fails: decrypt the 16-byte block with the AES key.
5. Check the decrypted plaintext for markers using a wider acceptance rule: `guidBytes[6] == 0x8D` (exact version match) AND `(guidBytes[8] & 0xC0) == 0x80` (RFC 9562 §4.1 variant range `0x80`–`0xBF`). The wider variant acceptance is necessary because the collision guard may have incremented the variant byte beyond `0x8D` during generation.
6. If the recovered variant byte is within the RFC 9562 variant range but less than `0x8D`, return INVALID. The generator never produces variant bytes below `0x8D`; such a value indicates tampering or an unrelated UUID.
7. If the recovered variant byte exceeds `0x8D`, perform backward collision-guard verification as specified in the Backward Verification Algorithm section. If verification fails, return INVALID.
8. If MAC verifies the decrypted plaintext, return the parsed SKEID with a secure-origin flag.
9. Otherwise, return INVALID.

The probability of a random or encrypted byte sequence coincidentally containing the marker bytes `0x8D8D` at the exact positions is approximately $1/65{,}536$. When this occurs, the algorithm gracefully falls back to the decryption path with no data corruption.

Secure SKEID 128-bit Specification

AES-256-ECB Single-Block Encryption

Table 5 shows the Secure SKEID byte layout after AES-256 encryption.

Table 5: Secure SKEID byte layout.

Field	Byte(s)	Width	Purpose
Ciphertext	0–15	128 bits	AES-256 encrypted SKEID (pseudorandom bytes)

As a single AES block, the entire SKEID is encrypted. No internal structure is visible to external consumers.

A Secure SKEID is produced by encrypting the entire 16-byte SKEID plaintext using AES-256 (National Institute of Standards and Technology, 2001) as a single-block cipher. Under the standard PRP assumption (Mouha, 2021), AES-256 applied to a single 128-bit block functions as a pseudorandom permutation (PRP), meaning every distinct plaintext maps to a distinct ciphertext and vice versa. A pseudorandom function (PRF) similarly produces outputs indistinguishable from random. The PRP/PRF switching lemma establishes a $2^{n/2}$ distinguishing bound for n-bit block ciphers (Bellare,

Krovetz & Rogaway, 1998). For AES with $n = 128$, a single-block PRP encryption is therefore indistinguishable from a random function up to approximately 2^{64} queries. This is beyond any practical identifier generation volume.

SKEID encryption always operates on exactly one block, the multi-block weakness of ECB mode does not apply. For a single block, ECB is mathematically identical to Cipher Block Chaining (CBC) with a zero initialization vector: $C = \mathrm{AES}(Key, P \oplus 0) = \mathrm{AES}(Key, P)$. No nonce is required, and there is no nonce-reuse vulnerability. ECB avoids the allocation and XOR overhead of a CBC initialization vector that would provide no additional security for single-block operations.

Quantum attacks on AES via Grover's algorithm reduce the effective security of AES-256 to 128 bits, which still remains computationally infeasible (Bonnetain, Naya-Plasencia & Schrottenloher, 2019).

Key Separation

The SKEID system uses two cryptographically independent keys derived from each key-ring entry. A MAC key is used for BLAKE3 keyed MAC (integrity verification) and an AES key is used for AES-256-ECB (confidentiality). The two keys must not be the same value. Implementations should derive both keys from a single master secret using a key derivation function or a deterministic hash chain that produces cryptographically independent outputs.

Collision Guard Mechanism

Without the collision guard, there exists a combined approximately $1/2^{48}$ probability that ciphertext coincidentally matches both the plaintext marker bytes and produces a valid MAC when interpreted as a plaintext SKEID. This would cause a Secure SKEID to be misclassified as a plaintext SKEID with incorrect data.

The collision guard eliminates this edge case by construction. During Secure SKEID generation, when SKEID Marker and MAC collision is detected in the ciphertext, the plaintext is regenerated with successive variant bytes from `0x8E` through `0xBF` (50 alternatives). Due to the AES avalanche effect, each variant produces completely different ciphertext. Termination is deterministic. The loop is bounded at 51 total attempts (1 primary + 50 alternatives), with an implementation-defined error on exhaustion (the reference implementation throws `JackpotException`). The probability of exhausting all 51 attempts is approximately $1/2^{48\times 51}$, which is vanishingly small.

Backward Verification Algorithm

During parsing, when a non-default variant byte V is recovered from the decrypted plaintext ($V > 0\text{x}8\text{D}$), the parse algorithm must verify that the previous variant ($V - 1$) genuinely triggered the collision guard. This is achieved by reconstructing the SKEID with variant $V - 1$, encrypting it, and checking whether the ciphertext exhibits the SKEID Marker and MAC collision. This single-step backward proof is sufficient by induction. If variant V is legitimate, then $V - 1$ must have collided, and $V - 1$'s legitimacy is either the base case ($V - 1 = 0\text{x}8\text{D}$, always legitimate) or proved by $V - 2$ having collided (which was already verified at generation time).

The result is a deterministic guarantee. No Secure SKEID produced by a compliant implementation can ever pass the plaintext parse path with a valid result.

Numeric Walkthrough

The following example illustrates the collision guard and backward verification with concrete values.

Setup: Consider a SKID with value `0x8BEB_C200_1204_0005` (timestamp = 400,000,000 ticks from epoch, equivalent to 100,000,000 seconds at 250ms precision, App ID = 18, App Instance ID = 1, Sequence = 5), entity type `0x0A` (entity type 10), epoch `0x00`, and key pair (K_{mac}, K_{aes}).

Step 1 SKEID Construction: The 16-byte SKEID buffer is constructed in big-endian order. Byte 0 receives the epoch (`0x00`), bytes 1–4 receive the sign-toggled SKID upper half (`0x8BEBC200` XOR `0x80000000` = `0x0BEBC200`), byte 5 receives the most significant byte of the SKID lower half, byte 6 receives the version marker (`0x8D`), byte 7 receives the entity type (`0x0A`), byte 8 receives the default variant marker (`0x8D`), bytes 9–11 receive the remaining SKID lower half bytes, and the BLAKE3 keyed MAC is computed and placed at bytes 12–15. The UUID is constructed from the 16-byte buffer in big-endian order per RFC 9562.

Step 2 Encryption and Collision Check: The 16-byte plaintext is encrypted with AES-256-ECB using K_{aes}, producing ciphertext C_1. The generator checks whether C_1 coincidentally has `0x8D` at byte position 6 and `0x8D` at byte position 8. In the overwhelming majority of cases (probability $\approx 1 - 1/65{,}536$), the ciphertext does not match, and C_1 is the final Secure SKEID.

Step 3 Collision Scenario: Suppose C_1 happens to exhibit `0x8D` at position 6 and `0x8D` at position 8, and furthermore the bytes at positions 12–15 coincidentally form a valid BLAKE3 MAC over the non-MAC bytes of C_1 when interpreted as a plaintext SKEID. This combined event has probability $\approx 1/2^{48}$.

When the collision guard activates, the plaintext variant byte (position 8) is changed from `0x8D` to `0x8E`. The BLAKE3 MAC is recomputed over the modified plaintext. The new plaintext is encrypted with the same K_{aes}, producing ciphertext C_2. Due to AES's avalanche property, even a single-bit change in plaintext produces ciphertext that differs in approximately 50% of its bits. The probability that C_2 also triggers a collision is again $\approx 1/2^{48}$, independent of the first collision. In practice, C_2 passes the check and becomes the final Secure SKEID.

Step 4 Backward Verification at Parse Time: When a consumer parses C_2 by decrypting with K_{aes}, the recovered plaintext reveals variant byte `0x8E` ($>$ 0x8D). The parser must verify that the escalation was legitimate.

1. Reconstruct the SKEID with variant `0x8D` (replacing `0x8E` and recomputing the MAC).
2. Encrypt that reconstruction with K_{aes} to obtain C_1.
3. Check that C_1 exhibits the marker-plus-MAC coincidence, confirming the collision that justified the escalation.
4. Since `0x8D` is the base case (always legitimate), the single backward step completes the proof.

If the backward check fails (the reconstructed C_1 does not exhibit the coincidence), the parser rejects the identifier as invalid. This prevents an attacker from crafting identifiers with arbitrary variant bytes.

Exact hex values, byte layouts, and round-trip assertions in this walkthrough are verified by the `PaperNumericWalkthroughTests` unit test suite in the reference implementation. Any code change that would invalidate these values produces a test failure.

Epoch and Extensibility

The 32-bit timestamp field stores 250-millisecond ticks, giving 2^{32} ticks per epoch half (equivalent to 2^{30} seconds). Each epoch half covers approximately 34 years. The sign bit doubles this to approximately 68 years per epoch while preserving monotonic sort order. The sign-bit toggle in the SKEID byte layout ensures that lexicographic comparison of UUID strings matches the signed chronological ordering of the underlying SKIDs. SKEID byte 0 carries an 8-bit epoch index, where each value selects a 2^{31}-second window starting from 2025-01-01, giving 256 epochs and approximately 17,421 years of total coverage.

Table 6 illustrates epoch addressing across the full epoch range.

Table 6: Epoch addressing.

Epoch Value	Start	End
`0x00`	January 1, 2025	January 19, 2093
`0x01`	January 19, 2093	February 7, 2161
⋮	⋮	⋮
`0xFF`	19378	19446

This two-level time addressing gives the identity system a multi-century lifespan without identifier collisions or sorting degradation. Epoch boundaries, total coverage, half-epoch boundary year, and sign-bit sort-order invariants in Table 6 are verified by the `PaperEpochAddressabilityTests` unit test suite in the reference implementation.

Defense-in-Depth Security Architecture

The SKEID system employs multiple verification layers. An attacker attempting to forge a valid identifier must defeat all applicable layers. Each layer is assumed to be independent of the others.

Table 7 enumerates the defense-in-depth security layers and their individual bypass probabilities.

Table 7: Defense-in-depth security layers.

Layer	Mechanism	Bypass Probability
1. AES-256 Encryption	PRP over 128 bits	2^{-128} without the key
2. BLAKE3 Keyed MAC	32-bit truncated	2^{-32} per attempt
3. Marker Detection	`0x8D8D` at fixed positions	2^{-16}
4. Entity Type Match	Must match the expected type	2^{-8}
5. Topology Validity	App ID and Instance ID must exist	Variable
6. Existence Probability	Forged SKID must reference a record	Variable
7. Rate Limiting	Restricts attempts per time period	Implementation defined

Even if an attacker guesses a ciphertext that, when decrypted, has valid markers, a valid MAC, a valid entity type, and valid topology, the resulting SKID must still correspond to an actual record in the database. The multiplicative combination of these barriers offers security far exceeding any individual layer. The layers are assumed to be operationally independent under secure key derivation. Each layer uses a different cryptographic primitive or validation mechanism. A formal proof of probabilistic independence is outside the scope of this paper (see Limitation 4).

MAC Truncation Analysis

The 32-bit truncation satisfies the minimum MacTag length specified in NIST SP 800-107 (Dang, 2012). The document notes that MacTags shorter than 64 bits are discouraged in Section 5.3.5. The 128-bit SKEID format constrains the available space for the MAC field, as the remaining bits carry the SKID, entity type, epoch, and marker bytes. This constraint is acceptable because the MAC is not the sole security mechanism. It operates within the defense-in-depth architecture described in Table 7, where AES-256 encryption, marker detection, entity type matching, and record existence probability provide additional verification layers.

Section 5.3.5 of the same document provides the general framework for assessing truncated MAC forgery risk. For a λ-bit MacTag with 2^t failed verifications allowed, the likelihood of accepting forged data is $(1/2)^{(\lambda-t)}$. Applying this to the 32-bit SKEID MAC ($\lambda = 32$): if a system permits 2^{12} (4,096) failed verification attempts before rotating the MAC key, the forgery likelihood is $(1/2)^{20}$, approximately one in a million. At 100 rate-limited attempts per second, this 2^{12} budget is exhausted in roughly 41 seconds.

SP 800-107 Rev. 1 was withdrawn in 2022 (NIST, 2022). Its successor, NIST SP 800-224 (Turan & Brandão, 2024), preserves the same truncated MAC analysis. SP 800-224 stipulates that tag lengths below 64 bits require careful risk analysis, which this section and the defense-in-depth architecture in Table 7 do.

Key Rotation and Key-Ring Fallback

A key-ring containing one or more key entries is required. Exactly one key is designated as the default (active) key at any time. Previous keys remain in the key-ring for parse fallback. All new SKEID and Secure SKEID generation uses the current default key. When parsing, the system attempts verification with the current default key first. If verification fails, it iterates through previous keys in reverse chronological order.

Key rotation does not require re-encrypting existing database records because the 64-bit SKID stored in the database contains no cryptographic material. Given the SKID and entity type, a new SKEID or Secure SKEID can be regenerated with the new key at any time. In the event of key compromise:

1. Add the compromised key to the key-ring with a "compromised" label.
2. Set a new key as the default.
3. Optionally regenerate SKEIDs from the immutable SKIDs.

AI-Assisted Research and Development

This subsection fulfills PeerJ CS's AI disclosure requirement for AI tools used as part of the research methodology.

The following AI tools and large language model (LLM) systems were used in the development of the reference implementation and the preparation of this research.

Software development, documentation, and testing: AI tools from Anthropic (Claude Opus/Sonnet series), Google (Gemini Pro/Flash series), OpenAI (o1, 4o), Qwen 3, and DeepSeek R1 were used across the DRN-Project development lifecycle, including the SKIDs implementation, as agentic coding assistants for code generation, refactoring, code review, test scaffolding, and documentation drafting. SKIDs were added to the DRN-Project roadmap on 19 November 2023. These AI tools were progressively adopted as they became available over the subsequent two-plus years of development. All generated code, tests, and documentation were reviewed, validated, and modified by the author.

Literature research: Google Gemini Deep Research was used to assist with surveying existing distributed identifier schemes and locating relevant prior work. All identified sources were independently verified by the author before citation.

Manuscript preparation: Claude Opus 4.6 and Gemini 3.1 Pro were used to draft, structure, and refine sections of this paper. All technical claims were verified against the implementation, and final editorial decisions were made by the author.

Prompt disclosure: AI interactions followed the following agentic workflow pattern. The author provided high-level task descriptions, architectural constraints, and acceptance criteria. The AI tools generated candidate implementations, which the author reviewed, modified, and validated. Following representative prompt categories included.

- "implement Secure SKEID generation with the following byte layout by using AES-256-ECB"
- "refactor sequence manager to use lock-free atomic operations"
- "write integration tests for Secure SKEID round-trip encryption using Testcontainers"
- "review this section for technical accuracy against the reference implementation and explain your judgement"

Due to the iterative, multi-session nature of the two-year development effort across multiple AI tools, exhaustive prompt logs are not available in a reproducible form. The prompts described above represent the general interaction patterns used throughout the project.

AI-assisted development workflows were guided by a structured behavioral framework enforcing security-first principles and systematic decision-making.

All AI-assisted outputs were reviewed, validated, and refined by the author. The architectural design, cryptographic choices, three-tier trust model, and all technical decisions are solely the work of the author. The author takes full responsibility for the correctness and originality of the work.

Reference Implementation

The reference implementation is provided as part of the open-source DRN.Framework (Kılıç, 2023), organized across four NuGet packages.

- **DRN.Framework.SharedKernel** defines interfaces (`ISourceKnownEntityIdOperations`), base classes (`SourceKnownEntity`), and parsed identity structs (`SourceKnownId`, `SourceKnownEntityId`).
- **DRN.Framework.Utils** implements cryptographic operations, timestamp management, sequence management (`SequenceManager<TEntity>` with lock-free atomic operations), and bit packing (`SourceKnownIdUtils`, `SourceKnownEntityIdUtils`).
- **DRN.Framework.EntityFramework** implements persistence integration including `SourceKnownIdValueGenerator` (EF Core value generator for automatic SKID assignment), `DrnSaveChangesInterceptor` (auto-assigns SKID and SKEID on insert), `DrnMaterializationInterceptor` (reconstructs full SKEID from stored SKID on database read), and `SourceKnownRepository<TContext, TEntity>` (generic repository with SKEID validation and cursor-based pagination).
- **DRN.Framework.Testing** implements integration and unit test infrastructure that provides Testcontainers orchestration for ephemeral PostgreSQL instances, auto-mocking through data attributes (`[DataInline]`, `[DataInlineUnit]`), and convention-based test contexts (`DrnTestContext`, `DrnTestContextUnit`) for low-friction, high-fidelity testing.

The implementation uses attribute-based dependency injection (`[Singleton<Type>]`, `[Scoped<Type>]`, `[Transient<Type>]`) and entity type attributes (`[EntityType(byte)]` for entity type registration) and integrates with Domain-Driven Design (Evans, 2003) patterns through the `SourceKnownEntity` abstract base class. The testing strategy and validation evidence for these packages are described in the Validation and Maturity section.

RESULTS

Reference Benchmark Environment

All benchmarks were conducted using BenchmarkDotNet (Akinshin et al., 2013) v0.15.8 on the following hardware and software configuration.

- **Hardware:** Apple M2, 1 CPU, 8 logical and 8 physical cores
- **Operating System:** macOS Tahoe 26.4 (25E246) [Darwin 25.4.0]
- **Runtime:** .NET 10.0.5 (10.0.526.15411), Arm64 RyuJIT armv8.0-a
- **SDK:** .NET SDK 10.0.201
- **Configuration:** OutlierMode=RemoveUpper, InvocationCount=262,144, IterationCount=120, WarmupCount=120

OutlierMode is set to RemoveUpper, a standard BenchmarkDotNet configuration that removes upper statistical outliers caused by garbage collector (GC) pauses and operating system (OS) scheduling jitter before computing summary statistics. This isolates measurements to the intrinsic operation cost.

The benchmark configuration uses 120 iterations with 262,144 invocations per iteration, enabling BenchmarkDotNet to compute error margins and standard deviations that approximate normal distribution. The invocation count of 262,144 matches the SKID system's 18-bit sequence counter cap (2^{18} identifiers per 250ms tick per instance). All invocations within a single iteration complete within the 250-millisecond sequence time scope, isolating the intrinsic computational cost of each operation without triggering the backpressure mechanism. A 250-millisecond `IterationSetup` delay between iterations allows the sequence time scope to reset, so each iteration starts from a clean state. Separate saturation benchmarks with `InvocationCount=786,432` (3 times the sequence cap), `IterationCount=40`, and `WarmupCount=40` (matching the iteration count) confirm the backpressure behavior. The saturation benchmark uses BenchmarkDotNet's default OutlierMode, retaining all measurements including upper outliers to capture the full backpressure distribution. Both configurations produce the same total invocation count ($120 \times 262{,}144 = 40 \times 786{,}432 = 31{,}457{,}280$). The saturation benchmark is configured to observe backpressure behavior rather than to produce directly comparable

absolute timings. SKID generation under saturation shows elevated mean times, reflecting the intermittent wait-for-next-tick pauses rather than the true per-operation cost. This validates the design. The sequence counter applies backpressure to preserve uniqueness guarantees under sustained load.

Performance Results

Table 8 presents the BenchmarkDotNet performance measurements for all SKID system operations. Mean, Error, and Standard Deviation values are reported in nanoseconds (ns) and rounded to a maximum of two decimal places. Error values represent the 99.9% CI margin computed by BenchmarkDotNet over iterations remaining after upper outlier removal.

Table 8: BenchmarkDotNet performance measurements (120 configured iterations, 262,144 invocations per iteration).

Operation	Mean (ns)	Error (ns)	StdDev (ns)	Allocated Memory
Random 64-bit integer generation	171.3	2.36	7.66	32 B
Random UUID V4 generation	354.1	3.64	11.81	0 B
Random UUID V7 generation	377.5	3.24	10.53	0 B
Timestamp manager current time generation	4.1	0.46	1.43	0 B
Sequence manager time-scoped ID generation	15.4	0.69	2.19	0 B
SKID generation	**35.3**	**1.21**	**3.92**	**0 B**
SKEID generation (with provided SKID)	219.0	3.25	10.56	0 B
SKEID generation (with SKID generation)	**230.3**	**3.45**	**11.20**	**0 B**
SKEID generation (with entity allocation)	248.3	3.87	12.58	192 B
Secure SKEID generation	**544.0**	**5.67**	**18.42**	**72 B**
SKEID parsing	223.6	2.87	9.31	0 B
Secure SKEID parsing	540.7	3.15	10.22	72 B
ToPlain (decryption only)	217.3	2.40	7.76	0 B
ToSecure (encryption only)	524.2	5.56	17.98	72 B

Performance Comparison with UUID

Secure SKEID generation at 544.0 ns takes approximately 1.4 times as long as UUID V7 at 377.5 ns. This overhead adds AES-256 encryption, BLAKE3 MAC integrity, entity type discrimination, and zero-lookup verification, none of which are provided by UUID V7. Secure SKEID can be converted to SKEID or SKID for the other desired identifier properties. The additional cost reflects approximately 314 ns of AES-256 single-block encryption beyond plaintext SKEID generation.

SKEID generation at 230.3 ns is approximately 1.6 times as fast as UUID V7 (377.5 ns) despite embedding additional metadata (entity type, epoch) and computing a BLAKE3 keyed MAC within the same 128-bit footprint. In the reference implementation, SKEID generation operates entirely on the stack (0 bytes allocated), avoiding garbage collection pressure. Implementations in other managed runtimes may exhibit different allocation profiles depending on stack allocation support.

SKID generation at 35.3 ns is more than 10 times as fast as UUID V7 generation at 377.5 ns and more than 10 times as fast as UUID V4 at 354.1 ns. This performance advantage stems from the deterministic bit-packing approach. SKID generation requires only an atomic counter increment, a cached timestamp read, and bitwise operations, with no cryptographic random number generation.

Throughput Analysis

The 18-bit sequence field caps generation at 262,144 identifiers per 250ms tick (1,048,576 per second) per instance. With 128 applications and 64 instances per application (8,192 total generators), the theoretical maximum system-wide throughput is approximately 8.6 billion identifiers per second. Full-throttle benchmarks confirm that the sequence manager applies backpressure when the per-instance limit is reached, preserving uniqueness under sustained load. Under saturation conditions (786,432 invocations per iteration), SKID generation mean rises from 35.3 ns to 610.0 ns because two-thirds of invocations exceed the per-tick sequence cap and must wait for the next 250-millisecond tick boundary. The saturation benchmark confirms that backpressure is precisely scoped to the sequence manager. All operations that internally generate a SKID (SKEID generation, Secure SKEID generation) show elevated means consistent with tick-boundary waiting, while operations that do not generate a SKID (parsing, tier conversion with a provided identifier) remain unaffected. This isolation validates the architectural separation between identifier generation and identifier processing (see supplementary file `supplementary-saturation-report.html`).

Throughput limits, generator topology, and system-wide capacity in this section are verified by the `PaperThroughputAnalysisTests` unit test suite in the reference implementation.

Storage Comparison

Table 9 compares storage requirements across identifier schemes.

Table 9: Storage requirements comparison.

Scheme	Primary Key Size	External ID Size	Ordered
SKID/SKEID	8 B	16 B	Yes
UUID V4	16 B	16 B	No
UUID V7	16 B	16 B	Yes
Snowflake	8 B	8 B	Yes
ULID	16 B	26 chars (Base32)	Yes
CUID2	24 chars	24 chars	No
KSUID	20 B	27 chars (Base62)	Yes
DB Sequence	4/8 B	4/8 B	Yes
Dual ID (int + UUID)	8 B + 16 B = 24 B	16 B	Integer only

At 8 bytes per SKID versus 16 bytes for UUID, storage cost halves, directly reducing archival and indexing overhead. Compared to KSUIDs at 20 bytes, storage cost is reduced by a factor of 2.5 per primary key. SKIDs as 64-bit integers provide superior B-tree index performance because smaller keys mean more keys per B-tree page, fewer page splits, and better cache utilization. The timestamp-leading layout means new records append to the end of the index, optimizing for append-heavy workloads. Cursor-based pagination is natively supported through the SKID's monotonic nature (`WHERE id > :lastId ORDER BY id LIMIT :pageSize`), eliminating offset-based pagination overhead. The dual-identifier pattern (integer primary key plus UUID external identifier) requires maintaining two separate columns with 24 bytes total per record. SKIDs eliminate this redundancy. The 8-byte SKID serves as the primary key, and the 16-byte SKEID or Secure SKEID is computed deterministically on demand without additional storage.

The computational cost of tier transformations is negligible. Converting between representations requires only sub-microsecond CPU operations. ToSecure (encryption) completes in 524.2 ns and ToPlain (decryption) in 217.3 ns (Table 8), with no I/O, no network round-trip, and no external dependency. Any alternative that relies on a secondary lookup, whether a database join for the dual-identifier pattern, a cache query, or a remote service call, incurs latency orders of magnitude higher. This makes the transformation cost effectively zero relative to any I/O-bound identifier resolution strategy.

DISCUSSION

Interpretation of Results

Benchmark results of the reference implementation demonstrate that the SKID system achieves competitive or superior performance compared to standard identifier generation while delivering substantially more functionality. The more than 10× speed advantage of SKID over UUID V7 stems from the deterministic generation approach. Bit packing from cached values is fundamentally less expensive than the cryptographic random number generation required by UUID schemes. Even the most expensive operation (Secure SKEID at 544.0 ns) completes in well under a microsecond, making it practical for high-throughput production systems.

Zero-allocation behavior in core SKID and SKEID operations is important for managed runtime environments (e.g., .NET, JVM) where garbage collection pauses can impact tail latency. By operating entirely on the stack, SKID generation avoids contributing to GC pressure even under sustained high-throughput scenarios.

Throughput scales linearly with the number of active generators. A single generator instance produces up to 1,048,576 identifiers per second (262,144 per 250ms tick times 4 ticks per second), bounded by the 18-bit sequence counter. With the default topology configuration supporting 8,192 concurrent generators (128 applications times 64 instances), the theoretical system-wide ceiling is approximately 8.6 billion identifiers per second. Most distributed applications will not approach this

ceiling. The backpressure mechanism ensures that overloaded generators degrade gracefully by waiting for the next tick rather than producing duplicates, a deterministic safety guarantee that probabilistic schemes cannot offer.

Comparison with Existing Work

Compared to UUID V7 (Davis, Peabody & Leach, 2024), the most directly comparable recent standard, SKIDs offer origin metadata, integrity verification (MAC), and confidentiality (encryption) that UUID V7 lacks. UUID V7 provides millisecond-precision timestamps while SKID uses 250-millisecond tick precision (see Design Trade-offs for the rationale and throughput analysis of this choice).

Snowflake (Twitter Engineering, 2010) shares the same 64-bit compact footprint as SKID, but the SKEID extension layer adds integrity and confidentiality that Snowflake lacks. Snowflake's 10-bit machine ID (1,024 machines) is comparable to SKID's combined 13-bit topology (7-bit App ID + 6-bit Instance ID), but SKEID adds explicit entity type discrimination. Both require coordination to assign machine/instance identifiers and both schemes converge on similar single-epoch durations (~68 years for SKID, ~69.7 years for Snowflake). However, Snowflake's epoch is non-extensible while SKEID can extend SKID with an 8-bit epoch index that provides 256 configurable epochs for approximately 17,421 years of total coverage. Lastly, Snowflake uses 41-bit millisecond timestamps while SKID achieves comparable coverage with 32-bit 250-millisecond ticks.

ULIDs (Feerasta, 2016) require twice the primary key storage (16 bytes versus 8 for SKID) and carry less embedded metadata. ULIDs use a 48-bit millisecond timestamp with ~8,919 years of coverage from the Unix epoch, while SKIDs achieve ~68 years per epoch with 256 configurable epochs for ~17,421 years total. ULIDs rely on 80 bits of cryptographic randomness for uniqueness within the same millisecond, a probabilistic guarantee, while SKIDs use deterministic topology-based partitioning. ULIDs use a 26-character Crockford Base32 encoding that sorts lexicographically, targeting string-first environments, but lack UUID format compatibility. SKEIDs are valid RFC 9562 Version 8 UUIDs. ULIDs provide neither integrity verification nor confidentiality.

KSUIDs (Segment, 2017) are the largest scheme in this comparison at 20 bytes per identifier (see Table 9). SKIDs provide a more compact representation (8/16 bytes) with richer metadata. KSUIDs use second-precision timestamps while SKIDs use 250-millisecond ticks. KSUIDs provide ~136 years coverage with 32-bit seconds and no extension mechanism. SKIDs provide ~68 years per epoch with 256 configurable epochs for ~17,421 years total. KSUID's 128-bit random payload trades storage efficiency for stronger per-identifier collision resistance without coordination. SKIDs achieve uniqueness through the topology-based partitioning scheme with deployment time coordination, which guarantees uniqueness by construction rather than by statistical improbability. Additionally, SKIDs provide entity type discrimination, integrity verification, and confidentiality, none of which KSUID offers. Even at the external-facing tier, SKEIDs (16 bytes) remain more compact than KSUIDs (20 bytes) while satisfying all six desired identifier properties. KSUIDs also lack UUID format compatibility, using a custom 27-character Base62 encoding that requires specialized tooling rather than standard UUID libraries available in every major programming language.

The original CUID (Elliott, 2012) was deprecated due to the security vulnerabilities inherent in k-sortable, timestamp-prefixed identifiers, as discussed in the Related Work section. Secure SKEIDs address this class of vulnerability directly. The AES-256 encryption layer transforms the entire UUID V8 payload into ciphertext indistinguishable from random bytes, preventing the information leakage that the CUID deprecation notice identifies as a systemic risk across timestamp-based and k-sortable identifier schemes.

CUID2 (Elliott, 2022) achieves uniqueness through SHA3-hashed entropy, a probabilistic guarantee. SKIDs instead offer deterministic uniqueness by construction. The CUID2 authors observe that "insecure ids can cause problems in unexpected ways, including unauthorized user account access, unauthorized access to user data, and accidental leaks of user's personal data which can lead to catastrophic effects, even in innocent-sounding applications like fitness run trackers" (Elliott, 2022). CUID2 addressed the security issues of its predecessor by replacing the k-sortable structure with SHA3-hashed entropy. CUID2 generates identifiers by combining multiple entropy sources and hashing them with SHA3, producing strong collision resistance but relying on the statistical properties of the hash function rather than structural guarantees. CUID2 also recommends a separate `createdAt`

database column for time-based sorting. That change introduces additional per-record storage overhead and a second indexed column, a trade-off structurally similar to the dual-identifier pattern (integer primary key plus UUID) that SKIDs eliminate by embedding the timestamp directly in the identifier.

SKIDs prefer context-dependent uniqueness over probabilistic uniqueness. This approach provides efficiency without sacrificing uniqueness guarantees.

Tables 10 and 11 compare the security posture of the SKEID system against standard and alternative identifier schemes.

Table 10: Security comparison of standard identifier schemes.

Threat	SKEID / Secure SKEID	UUID V4	UUID V7	Snowflake
Enumeration	MAC + optional AES	122-bit random	Timestamp-ordered	Timestamp-ordered
Forgery	MAC verification	No detection	No detection	No detection
Info leakage	AES-256 encrypted	None (random)	Timestamp visible	TS + worker visible
Cross-type confusion	Entity type check	Possible	Possible	Possible

Table 11: Security comparison of alternative identifier schemes.

Threat	SKEID / Secure SKEID	ULID	CUID2	KSUID
Enumeration	MAC + optional AES	TS + 80-bit random	Hash-based	TS + 128-bit random
Forgery	MAC verification	No detection	No detection	No detection
Info leakage	AES-256 encrypted	Timestamp visible	None (hashed)	Timestamp visible
Cross-type confusion	Entity type check	Possible	Possible	Possible

Threat Analysis

To evaluate the security posture of the SKEID system systematically, we apply a threat analysis informed by the STRIDE categories (Shostack, 2014) (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to the three-tier identity model.

Spoofing (Identifier Fabrication): An attacker who intercepts a Secure SKEID cannot derive the plaintext SKEID without the AES-256 key. Forging a valid Secure SKEID requires producing ciphertext that, when decrypted, yields valid markers, a correct BLAKE3 MAC, a valid entity type, and a SKID corresponding to an existing record. The multiplicative barrier across layers 1–6 in Table 7 makes this computationally infeasible. For plaintext SKEIDs within trusted environments, the BLAKE3 MAC prevents identifier fabrication without the MAC key.

Tampering (Identifier Modification): Any modification to a Secure SKEID produces different plaintext upon decryption (AES-256 PRP property), invalidating the MAC with probability $1 - 2^{-32}$. For plaintext SKEIDs, modifying any byte invalidates the MAC because the MAC is computed over the full 16-byte buffer.

Repudiation: The SKEID system does not provide non-repudiation in the cryptographic sense because it does not use digital signatures. However, the embedded topology metadata (App ID, App Instance ID, Entity Type) and timestamp provide forensic traceability. For a given SKEID, the originating application, instance, entity type, and approximate creation time can be determined.

Information Disclosure: Plaintext SKEIDs deliberately expose metadata (timestamp, entity type, topology) within trusted environments where this information supports routing and validation. For external consumers, Secure SKEIDs encrypt the entire identifier using AES-256-ECB, which functions as a pseudorandom permutation on the single 128-bit block. The ciphertext is computationally indistinguishable from random bytes without the key, preventing timestamp analysis, entity type inference, and generation pattern discovery.

Denial of Service: The 18-bit sequence counter limits generation to 262,144 identifiers per 250ms tick (1,048,576 per second) per instance. Exceeding this rate triggers backpressure (the generator waits for the next tick), which is a deliberate safety mechanism rather than a vulnerability. An attacker cannot cause identifier exhaustion across the system because each of the 8,192 possible generators operates independently. Parse operations (both encrypted and plaintext) are bounded-time with no external dependencies, preventing parse-amplification attacks.

Elevation of Privilege (Cross-Entity Confusion): The 8-bit entity type discriminator prevents cross-entity attacks where an identifier for one entity type (e.g., User with entity type 1) is submitted as another (e.g., AdminRole with entity type 5), potentially granting unintended access or administrative privileges. Because the MAC includes the entity type in its computation, a valid SKEID for entity type 1 cannot pass MAC verification when checked against entity type 5.

Validation and Maturity

Active development of the SKID system began in November 2023 as part of the DRN.Framework. The framework has undergone continuous refinement across versioned releases (v0.1.0 through v0.8.1). Each release is validated through integration and unit tests built on DRN.Framework.Testing. Performance is measured separately with BenchmarkDotNet.

The framework's correctness is validated through multiple test and analysis tiers.

- **Unit tests:** In-memory tests covering SKID generation, SKEID construction with MAC verification, and Secure SKEID encryption/decryption round-trips.
- **Paper-verification tests:** Dedicated suites (`PaperNumericWalkthroughTests`, `PaperEpochAddressabilityTests`) assert the exact numeric values, byte layouts, epoch boundaries, lexicographic ordering across epoch halves, and sort-order invariants presented in this paper. Any implementation change that would invalidate a paper claim produces a test failure.
- **Integration tests:** Testcontainers-based tests provision ephemeral PostgreSQL instances, exercising Entity Framework Core value generation, interceptors, and cursor-based pagination with SKID ordering.
- **End-to-end validation:** Test applications validated behavior across the three tiers: SKID assignment during entity creation, SKEID exposure within trusted application-to-application communication, and Secure SKEID generation for external API responses. These applications confirm that bidirectional transformations preserve data integrity across the complete round-trip.
- **Static analysis:** GitHub CodeQL and SonarCloud continuously scan the codebase for security vulnerabilities, code quality issues, and potential bugs. The framework maintains zero critical or high-severity findings.

The four NuGet packages have been published on NuGet.org since the initial framework release and are available for community review and adoption. They are published through an automated GitHub Actions CI/CD pipeline that runs the complete test suite before each release. Therefore, no regression reaches published consumers.

Design Trade-offs

250-millisecond tick versus millisecond precision: SKIDs use 250-millisecond tick timestamps (32 bits, four ticks per second) instead of millisecond-precision (48 bits in UUID V7). This sub-second quantization provides finer ordering than second-precision schemes while preserving sufficient bits for application topology and sequence fields within 64 bits. The 18-bit sequence (262,144 identifiers per tick; 1,048,576 per second) delivers high throughput within each tick. Customized domain-tailored bit layouts can be more suitable for specific use cases.

Clock synchronization versus drift protection: The SKID system delegates clock synchronization to the operating system. Accurate timekeeping through Network Time Protocol (NTP) (Mills et al., 2010), Precision Time Protocol (PTP) (IEEE, 2020), or equivalent protocols is an infrastructure responsibility outside the scope of the identifier generator. The clock drift protection mechanism described in the SKID specification handles only the consequences of clock jumps. Minor backward drifts freeze the timestamp until the wall clock catches up, while drifts beyond the freeze threshold force an application restart with a new instance identifier. This separation of concerns keeps the identifier generator simple and deterministic, relying on the operating system to maintain monotonic time accuracy.

32-bit truncated MAC versus full MAC: The 4-byte MAC is shorter than typical MACs (16–32 bytes) but is one layer in a defense-in-depth strategy. The MAC is not the sole security mechanism. It works in concert with AES encryption, marker detection, entity type matching, and record existence probability.

Coordination versus probabilistic uniqueness: The SKID system requires deployment-time coordination to assign `AppId` (7 bits, up to 128 applications) and `AppInstanceId` (6 bits, up to 64 instances per application). This stands in contrast to UUID V4, UUID V7, and CUID2, which achieve uniqueness without any coordination through cryptographic random number generation. The coordination cost is an intentional architectural trade-off. Deterministic uniqueness by construction eliminates the residual collision probability inherent in probabilistic schemes, enables zero-lookup verification through MAC and topology metadata, and allows the compact 64-bit representation that probabilistic schemes cannot achieve at equivalent uniqueness guarantees.

Topology field sizing: The 7-bit application field (128 applications) and 6-bit instance field (64 instances per application) reflect practical operational boundaries rather than arbitrary bit allocation. Beyond 128 independently deployed applications, coordination complexity becomes infeasible due to the increase in the number of intercommunication paths ($n(n-1)/2$) (Jr., 1995, chap. 2). Well-designed systems mitigate this through bounded contexts (Evans, 2003) rather than unbounded application proliferation. Similarly, beyond approximately 10 concurrently active instances per application, resource contention on shared infrastructure (databases, message brokers) typically becomes the throughput bottleneck before the identifier generator itself. The 6-bit field accommodates up to 64 instance identifiers, providing headroom for operational needs such as rolling deployments and application restarts triggered by clock drift protection, where the restarting instance must acquire a new instance identifier to prevent duplicate identifiers. These limits are defaults for the general case. The system is intentionally optimized for stable, bounded distributed application topologies. Domain-tailored bit layouts (see Future Work, item 3) remain available for specialized deployments requiring different trade-off profiles.

.NET Guid mixed endianness: The .NET `Guid` struct uses a mixed-endian internal representation. The first three fields (`Data1`, `Data2`, `Data3`) are stored in little-endian order, while the remaining eight bytes (`Data4`) are stored sequentially. This internal layout is an implementation artifact of the Windows COM heritage and differs from the big-endian (network byte order) specified by RFC 9562. The SKEID implementation uses the .NET 8+ constructors `Guid(ReadOnlySpan<byte>, bigEndian: true)` and `Guid.ToByteArray(bigEndian: true)` to ensure that byte arrays are always constructed and interpreted in RFC 9562 big-endian format, regardless of the platform's internal `Guid` representation. The string representation produced by `Guid.ToString()` is always big-endian and RFC 9562 compliant.

Limitations

1. **Deployment-scoped uniqueness:** SKIDs are unique within a deployment (same epoch, same key-ring) but not globally unique across independent deployments. Applications requiring cross-deployment identity merging must include deployment identifiers in their routing logic.

2. **Topology limits and coordination requirement:** The default profile supports at most 128 applications times 64 instances = 8,192 distinct generators. Topology assignment (AppId and AppInstanceId) requires deployment-time coordination, unlike UUID V4/V7/CUID2 which achieve uniqueness without coordination. The rationale for this specific field sizing is discussed in Design Trade-offs. Reference implementation aims to provide a coordination application called `Nexus` for this purpose.

3. **Key dependency:** SKEIDs and Secure SKEIDs require key material for generation and parsing. Loss of key material makes existing SKEIDs unparseable, though the underlying 64-bit SKIDs in the database remain valid and recoverable.

4. **Security analysis scope:** The security analysis in this paper uses STRIDE-based threat modeling and quantitative probability analysis within the defense-in-depth architecture. It does not provide formal cryptographic reductions (e.g., proving that the composition of AES-PRP encryption, BLAKE3 MAC, and the collision guard mechanism achieves the intended security goals under standard assumptions). The Secure SKEID scheme inherits PRP security from the AES block cipher, and the uniqueness of SKEID plaintexts prevents ciphertext equality leakage. A formal compositional security proof is left as future work. The reference implementation provides test vectors in its test suites enabling independent verification.

5. **Epoch-scoped storage efficiency:** The 8-byte SKID storage efficiency claim applies within a single epoch (~68 years). The 64-bit SKID encodes a 32-bit timestamp relative to the configured epoch but does not carry the epoch index itself. A database spanning multiple epochs has the following options.

    1. A composite key storing the epoch byte separately alongside the SKID. This adds per-row overhead that negates the 8-byte claim.
    2. Migration to the 16-byte SKEID, which includes the epoch at byte 0. This also negates the 8-byte claim.
    3. Epoch-partitioned storage, where the table or archive name implicitly carries the epoch context (e.g., `Entity_Epoch0`, `Entity_Epoch1`). This preserves 8-byte rows by encoding the epoch in the partition structure rather than in each record.

Since 68 years exceeds the operational lifespan of most modern software systems, this constraint does not diminish practical utility.

6. **Secure SKEID UUID format compliance:** A Secure SKEID is a valid UUID in length and string format (32 hexadecimal digits in `8-4-4-4-12` grouping) and is accepted by standard data-type parsers (e.g., PostgreSQL `uuid` columns, .NET `Guid`, Java `UUID`). However, AES-256 encryption replaces the plaintext Version and Variant marker bytes with pseudorandom ciphertext. The encrypted output will almost never contain the `0x8` version nibble and `10xx` variant bits required by RFC 9562 Section 4.1 and Section 5.8. Strict RFC 9562 validators will reject a Secure SKEID as having an unknown version. Systems requiring strict RFC 9562 compliance at the validator level should use plaintext SKEIDs or accept Secure SKEIDs as opaque 128-bit values.

Future Work

Several extensions are planned or under consideration.

1. **Epoch transition:** The mechanism for transitioning from epoch `0x00` to epoch `0x01` is not specified. Given that epoch `0x00` spans until approximately 2093 CE, this is left as future work.
2. **Key-ring rotation with multiple active key versions and graceful rollover:** The reference implementation currently uses a single key pair. The key-ring model and fallback algorithm are left as future work.
3. **Domain-tailored bit layouts:** Custom SKID profiles with different field widths (e.g., nanosecond or picosecond timestamp precision, single-year epoch windows for mission-scoped campaigns) would enable research workflows with different trade-off requirements. Configurable custom profiles are not standardized yet but under consideration.
4. **IETF standardization:** A draft specification prepared in Internet-Draft format (Kılıç, 2026) has been authored and is available for independent review, containing complete algorithms, CDDL definitions, and reproducible test vectors. Formal submission to the IETF would enable interoperability testing and community review.
5. **Formal compositional security proof:** A formal cryptographic reduction proving that the composition of single-block AES-PRP encryption, BLAKE3 MAC, and the collision guard mechanism achieves the intended security goals under standard assumptions would strengthen the security analysis beyond the STRIDE-based threat modeling and quantitative probability analysis provided in this paper.
6. **Cross-platform benchmarks:** The current benchmarks were conducted on ARM64 (Apple M2) with .NET 10. Benchmarks on x86-64 architectures and other language runtimes would validate portability of the performance characteristics.
7. **Multi-language implementations:** The SKID design is language-agnostic. Implementations in Javascript, Java, Go, Rust, Python and other languages would expand the ecosystem and validate the specification's portability. These implementations are left to the open-source community.

CONCLUSIONS

This paper presented Source Known Identifiers (SKIDs), a three-tier identity system that simultaneously satisfies the proposed desired identifier properties. Based on our literature survey, no existing identifier scheme combines all six properties within a unified system.

The three-tier architecture aligns SKID (64-bit, database), SKEID (128-bit, trusted internal), and Secure SKEID (128-bit, external) with the trust boundaries commonly found in distributed applications. Deterministic bidirectional transformations enable direct conversion between tiers. The defense-in-depth security architecture combines multiple verification layers to provide security far exceeding any individual mechanism.

Benchmark results show that SKID and SKEID achieve superior performance compared to UUID generation in the reference implementation. Secure SKEID remains competitive while encapsulating desired identifier properties with AES-256 encryption at approximately 1.4 times the generation cost of UUID V7.

The open-source reference implementation, published as NuGet packages with integration and unit test coverage, is available for community review and adoption. A draft specification prepared in Internet-Draft format is planned as future work to enable cross-platform implementation and interoperability testing.

ACKNOWLEDGMENTS

This work received no external financial support. GitHub CodeQL and SonarCloud provided static analysis and code quality scanning that strengthened the security posture of the implementation. CodeRabbit provided AI code review that contributed to the overall code quality.

DATA AVAILABILITY

The source code for DRN-Project is available at https://github.com/duranserkan/DRN-Project and archived on Zenodo (DOI: 10.5281/zenodo.19240397). The archived release (v0.9.1) is available at https://github.com/duranserkan/DRN-Project/tree/v0.9.1. Raw benchmark data (CSV) and rendered reports (HTML) are included as six supplementary files covering the primary benchmark, saturation benchmark, and hash comparison benchmark.

COMPETING INTERESTS

The author declares no competing interests.

FUNDING

This work received no external financial support.

AUTHOR CONTRIBUTIONS

Duran Serkan Kılıç conducted the research, conceived the design, implemented the software, conducted the benchmarks, and wrote the manuscript.

REFERENCES

Akinshin A, others. 2013. BenchmarkDotNet: Powerful .NET library for benchmarking.

Bellare M, Krovetz T, Rogaway P. 1998. Luby-Rackoff Backwards: Increasing Security by Making Block Ciphers Non-Invertible. In: *Advances in cryptology – EUROCRYPT '98*. LNCS. 266–280. DOI: 10.1007/BFb0054132.

Bonnetain X, Naya-Plasencia M, Schrottenloher A. 2019. Quantum security analysis of AES. *IACR Transactions on Symmetric Cryptology* 2019:55–93. DOI: 10.13154/tosc.v2019.i2.55-93.

Dang Q. 2012. *Recommendation for Applications Using Approved Hash Algorithms*. NIST. DOI: 10.6028/NIST.SP.800-107r1.

Davis K, Peabody B, Leach P. 2024. *Universally Unique IDentifiers (UUIDs)*. IETF. DOI: 10.17487/RFC9562.

Elliott E. 2012. CUID: Collision-resistant ids (deprecated).

Elliott E. 2022. CUID2: Secure, collision-resistant ids.

Evans E. 2003. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Feerasta A. 2016. ULID: Universally Unique Lexicographically Sortable Identifier.

IEEE. 2020. *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. DOI: 10.1109/IEEESTD.2020.9120376.

Jr. FPB. 1995. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.

Kılıç DS. 2023. DRN-Project: Distributed Reliable .Net. DOI: 10.5281/zenodo.19240397.

Kılıç DS. 2026. Source Known Identifiers (SKIDs): Technical Specification Draft. DOI: 10.5281/zenodo.19240397.

Mills D, Martin J, Burbank J, Kasch W. 2010. *Network Time Protocol Version 4: Protocol and Algorithms Specification*. IETF. DOI: 10.17487/RFC5905.

MITRE Corporation. 2025a. CWE-639: Authorization Bypass Through User-Controlled Key.

MITRE Corporation. 2025b. CWE-345: Insufficient Verification of Data Authenticity.

Mouha N. 2021. *Review of the Advanced Encryption Standard*. NIST. DOI: 10.6028/NIST.IR.8319.

National Institute of Standards and Technology. 2001. *Advanced Encryption Standard (AES)*. NIST. DOI: 10.6028/NIST.FIPS.197-upd1.

NIST. 2022. Withdrawal of NIST Special Publication 800-107 Revision 1.

O'Connor J, Aumasson J-P, Neves S, Wilcox-O'Hearn Z. 2020. BLAKE3: One function, fast everywhere.

Ruggles R, Brodie H. 1947. An Empirical Approach to Economic Intelligence in World War II. *Journal of the American Statistical Association* 42:72–91. DOI: 10.1080/01621459.1947.10501915.

Segment. 2017. KSUID: K-Sortable Globally Unique IDs.

Shostack A. 2014. *Threat Modeling: Designing for Security*. Wiley.

Turan MS, Brandão LTAN. 2024. *Keyed-Hash Message Authentication Code (HMAC): Specification of HMAC and Recommendations for Message Authentication*. NIST. DOI: 10.6028/NIST.SP.800-224.ipd.

Twitter Engineering. 2010. Announcing Snowflake.